



รายงานโปรเจกต์ CarTally

จัดทำโดย

6610110074 นายโชติมนัส ชนะกุล

6610110190 นายพชรพล เกตุแก้ว

6610110424 นายคมสัน คงผล

คณะวิศวกรรมศาสตร์ สาขาวิศวกรรมคอมพิวเตอร์

เสนอ

อาจารย์ รศ.ดร. พิชญา ตันทัย

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา

240-229 SOFTWARE DEFINED ARCHITECTURE ENGINEER MODULE

ภาคเรียนที่ 2 ปีการศึกษา 2567

คณะวิศวกรรมศาสตร์ มหาวิทยาลัยสงขลานครินทร์ วิทยาเขตหาดใหญ่

คำนำ

รายงานเล่มนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของการศึกษาวิชา **240-229 SOFTWARE DEFINED ARCHITECTURE ENGINEER MODULE** ซึ่งเป็นวิชาที่มุ่งเน้นการเรียนรู้แนวคิดและหลักการของสถาปัตยกรรมระบบที่กำหนดโดยซอฟต์แวร์ (Software-Defined Architecture) รวมถึงการออกแบบและพัฒนาโซลูชันทางวิศวกรรมที่สามารถปรับเปลี่ยนได้ตามความต้องการของระบบ โดยมีวัตถุประสงค์เพื่อให้ผู้เรียนมีความเข้าใจเกี่ยวกับโครงสร้างของระบบที่ควบคุมด้วยซอฟต์แวร์ การพัฒนาแอปพลิเคชันที่สามารถปรับขยายได้ และการบริหารจัดการทรัพยากรของระบบให้เกิดประสิทธิภาพสูงสุด

ผู้จัดทำหวังเป็นอย่างยิ่งว่า รายงานเล่มนี้จะเป็นประโยชน์กับผู้อ่าน โดยเฉพาะนักเรียน นักศึกษา หรือผู้ที่สนใจในด้าน **Software-Defined Architecture** หากมีข้อเสนอแนะหรือข้อผิดพลาดประการใด ผู้จัดทำขอน้อมรับไว้และขออภัยมา ณ ที่นี้ด้วย

กลุ่ม Dtect

วันที่ 04/03/2568

สารบัญ

บทนำ.....	1
การทำงานของโปรเจกต์.....	2
ภาพรวมของโปรเจกต์	2
การทำงานฝั่ง Web Client.....	4
Multiuser และ Security ในระบบ Login.....	5
การทำงานฝั่ง Server และ Database	10
การทำงานฝั่ง AI	11
การใช้งาน Cloud เพื่อ Deploy เว็บไซต์	13
การใช้งาน Cloud เพื่อ Deploy เซิร์ฟเวอร์.....	17
การใช้งาน Git Actions เพื่อทำ CI/CD.....	19
การใช้งาน Cloud เพื่อส่งข้อมูลพาหนะรายวัน	22
การใช้งาน Cloud เพื่อสังเกตการณ์ค่าใช้จ่ายจากนั้นแจ้งเตือนไปยังอีเมล	26
การประมวลผลข้อมูลด้วย Multithreading เพื่อการจัดการยานพาหนะแบบเรียลไทม์	28
สรุปรายการเทคนิคที่ใช้ในโปรเจกต์	29
Category I.....	29
Category II.....	29
Category III.....	29
Category IV.....	30
Category V	30
Category VI.....	31

สารบัญรูปภาพ

รูป 1 Use Case Diagram ของแอปพลิเคชัน	2
รูป 2 Diagram ภาพรวมของแอปพลิเคชัน	3
รูป 3 เว็บไซต์หน้า Landing	4
รูป 4 เว็บไซต์หน้าเข้าสู่ระบบ	5
รูป 5 เว็บไซต์หน้าลงทะเบียน	5
รูป 6 การแสดงผลเว็บไซต์เมื่อไม่มีสิทธิ์เข้าถึง	6
รูป 7 เว็บไซต์หน้าวิดีโอย้อนหลัง.....	6
รูป 8 หน้า Dashboard แสดงวิดีโอ.....	7
รูป 9 หน้า Dashboard แสดง Log ของพาหนะที่ตรวจจับได้ล่าสุด.....	7
รูป 10 หน้าวิดีโอย้อนหลังแสดงรายละเอียดวิดีโอล่าสุดและพาหนะที่ตรวจจับได้	8
รูป 11 แสดงรายละเอียดวิดีโอ	8
รูป 12 หน้าการวิเคราะห์	9
รูป 13 การแสดงผลเมื่อรัน main.py	11
รูป 14 รายละเอียดการแสดงผลต่างๆเมื่อรัน main.py.....	12
รูป 15 ไฟล์ main.tf ที่ใช้สร้าง VM.....	13
รูป 16 รายละเอียด VM ที่ได้จาก Terraform	13
รูป 17 ไฟล์ dtech ซึ่งเป็น config ของ nginx	14
รูป 18 ข้อมูลหลังจาก build ใน /var/www/client	14
รูป 19 การแสดงผลเมื่อใช้ pm2 ถูกต้อง	14
รูป 20 เว็บไซต์หลังจากทำทุกขั้นตอน	15
รูป 21 รายละเอียดการสร้าง Snapshot ของ VM.....	15
รูป 22 ผลการสร้าง Snapshot ของ VM.....	16
รูป 23 Docker image ที่อยู่ใน Docker Hub	17
รูป 24 GKE Cluster.....	17
รูป 25 แผนภาพ Traffic ของเซิร์ฟเวอร์	18

รูป 26 Git Actions ใน Repository	19
รูป 27 รายละเอียด Deploy Next.js to Google Cloud VM.....	19
รูป 28 ไฟล์ deploy.yml สำหรับ workflow.....	20
รูป 29 รายละเอียด Test Workflow	20
รูป 30 ไฟล์ test.yml สำหรับ workflow.....	21
รูป 31 แผนภาพการใช้งาน Cloud สำหรับการส่งข้อมูลพาหนะรายวัน	22
รูป 32 การใช้งาน Cloud Functions ชื่อ daily-vehicle-detected-log	22
รูป 33 การใช้งาน Cloud Scheduler เพื่อส่งงานฟังก์ชัน	23
รูป 34 ตัวอย่างไฟล์ Excel ที่ใช้ในการบันทึก.....	24
รูป 35 Bucket ของ Google Cloud Storage.....	24
รูป 36 ข้อมูลอีเมลที่ส่งให้ปลายทาง.....	25
รูป 37 แผนภาพแสดงลำดับการทำงานของ การส่งอีเมล	26
รูป 38 แสดง topic	26
รูป 39 แสดงข้อมูลที่ถูกรับผ่าน PUB/SUB	27
รูป 40 อีเมลที่ได้รับการแจ้งเตือน	27
รูป 41 โค้ด services สำหรับเรียกใช้ multithread	28

บทนำ

ในปัจจุบัน การตรวจจับยานพาหนะและการวิเคราะห์การจราจรมีความสำคัญอย่างมากในหลายด้าน เช่น การบริหารจัดการจราจร การตรวจจับการละเมิดกฎจราจร และการเพิ่มประสิทธิภาพของระบบขนส่งสาธารณะ ด้วยเทคโนโลยี AI และ Machine Learning ที่ก้าวหน้า ระบบตรวจจับยานพาหนะแบบเรียลไทม์จึงสามารถช่วยในการบริหารจัดการจราจรเป็นไปอย่างมีประสิทธิภาพมากขึ้น

CarTally เป็น Yolo Vehicle Detection Web ซึ่งเป็นเว็บแอปพลิเคชันที่พัฒนาขึ้นเพื่อรองรับการตรวจจับยานพาหนะที่เข้าออกในแต่ละเลน โดยใช้โมเดล YOLO (You Only Look Once) ซึ่งเป็นโมเดล deep learning ที่สามารถตรวจจับวัตถุได้อย่างรวดเร็วและแม่นยำ นอกจากนี้ ระบบยังสามารถคำนวณความเร็วของรถยนต์โดยอาศัยการประมวลผลภาพวิดีโอ และแสดงผลแบบ real-time ผ่านเว็บไซต์ เพื่อเพิ่มประสิทธิภาพและความสะดวกในการใช้งาน เทคโนโลยี Cloud ได้ถูกนำมาใช้ในโครงสร้างของระบบ ดังนี้

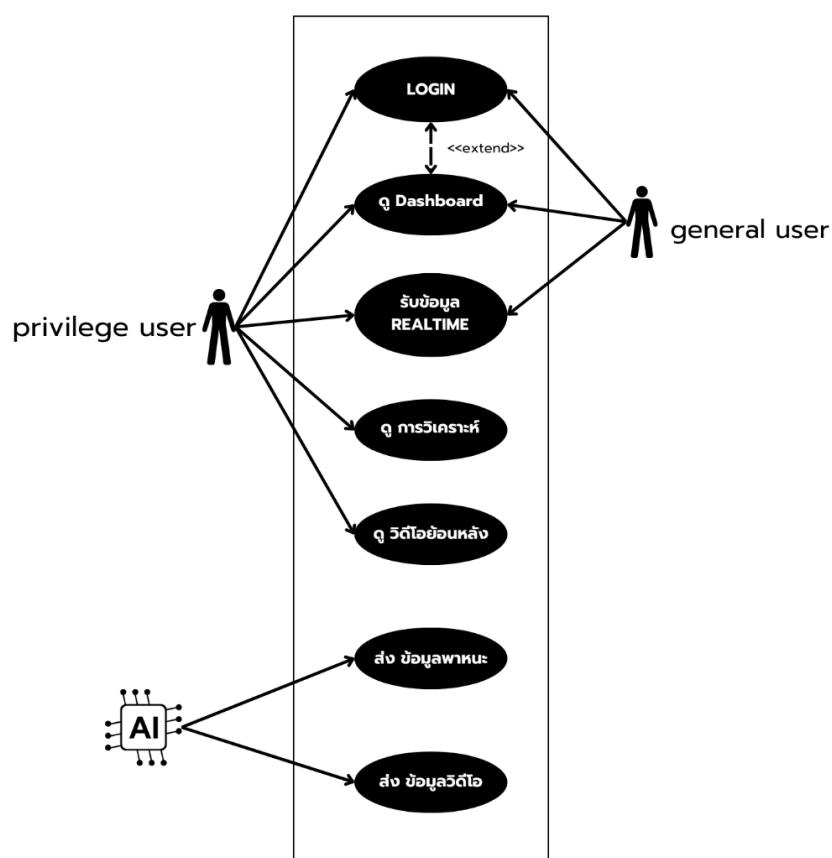
- **Compute Engine:** ใช้สำหรับแสดงผลเว็บไซต์ โดยโฮสต์ Frontend และ Backend ของระบบ เพื่อให้ผู้ใช้สามารถเข้าถึงข้อมูลและดูผลลัพธ์ได้อย่างรวดเร็ว
- **Kubernetes Engine:** ช่วยจัดการการปรับขยาย (scalability) ของระบบ โดยรัน containerized applications เพื่อให้สามารถรองรับการประมวลผลที่เพิ่มขึ้นได้อย่างมีประสิทธิภาพ
- **Cloud Storage:** ใช้สำหรับจัดเก็บข้อมูลพาหนะที่ตรวจจับได้ในแต่ละวัน
- **Cloud SQL:** ใช้เป็นฐานข้อมูลสำหรับจัดเก็บข้อมูลยานพาหนะ ข้อมูลการคำนวณความเร็ว และข้อมูลจราจรในรูปแบบที่มีโครงสร้าง ทำให้การเข้าถึงข้อมูลมีประสิทธิภาพ
- **Cloud Functions:** ใช้สำหรับประมวลผลฟังก์ชันขนาดเล็ก เช่น การสร้างไฟล์ Excel ที่มีข้อมูลของพาหนะในแต่ละวิดีโอ
- **Cloud Monitoring:** ใช้ติดตามประสิทธิภาพของระบบ พร้อมแจ้งเตือนเมื่อมีปัญหา เช่น การใช้จ่ายเงินที่เกินขีดจำกัด

การทำงานของโปรเจกต์

ภาพรวมของโปรเจกต์

CarTally ประกอบไปด้วย 4 ส่วนหลัก ได้แก่ Web Client, Server, AI และ Database ซึ่งแต่ละส่วนมีบทบาทสำคัญและทำงานร่วมกันเพื่อให้ระบบสามารถทำงานได้อย่างสมบูรณ์แบบ โดยมีรายละเอียดดังนี้:

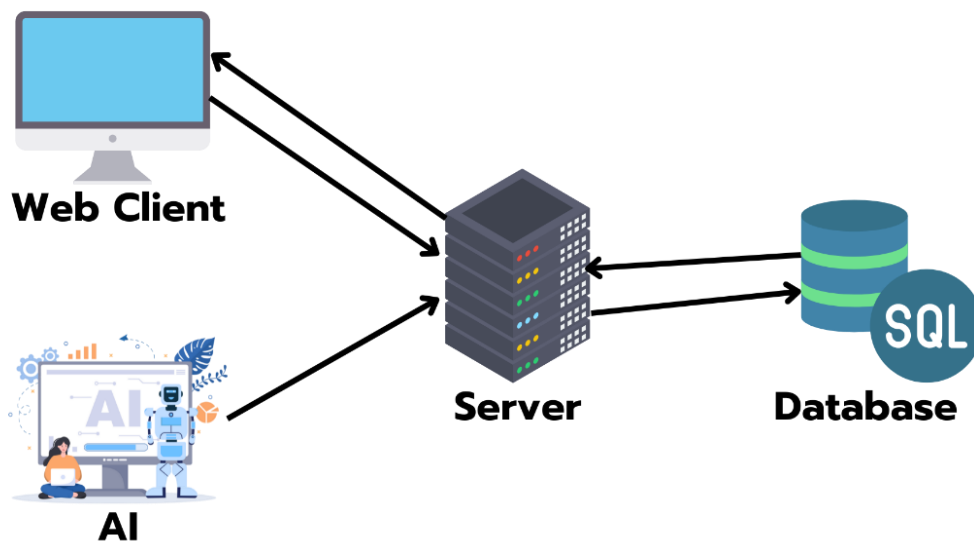
- **Web Client:** เป็นส่วนติดต่อผู้ใช้ (Frontend) ที่พัฒนาโดยใช้ Next.js เพื่อให้ผู้ใช้สามารถดูผลลัพธ์ของการตรวจจับยานพาหนะและข้อมูลที่เกี่ยวข้องแบบ real-time ผ่านเว็บเบราว์เซอร์



รูป 1 Use Case Diagram ของแอปพลิเคชัน

- **Server:** ทำหน้าที่เป็นตัวกลางในการรับ-ส่งข้อมูลระหว่าง Web Client, AI Model และ Database โดยใช้ Node.js พร้อมรองรับการเชื่อมต่อผ่าน API เพื่อให้สามารถรับและส่งข้อมูลได้อย่างมีประสิทธิภาพ

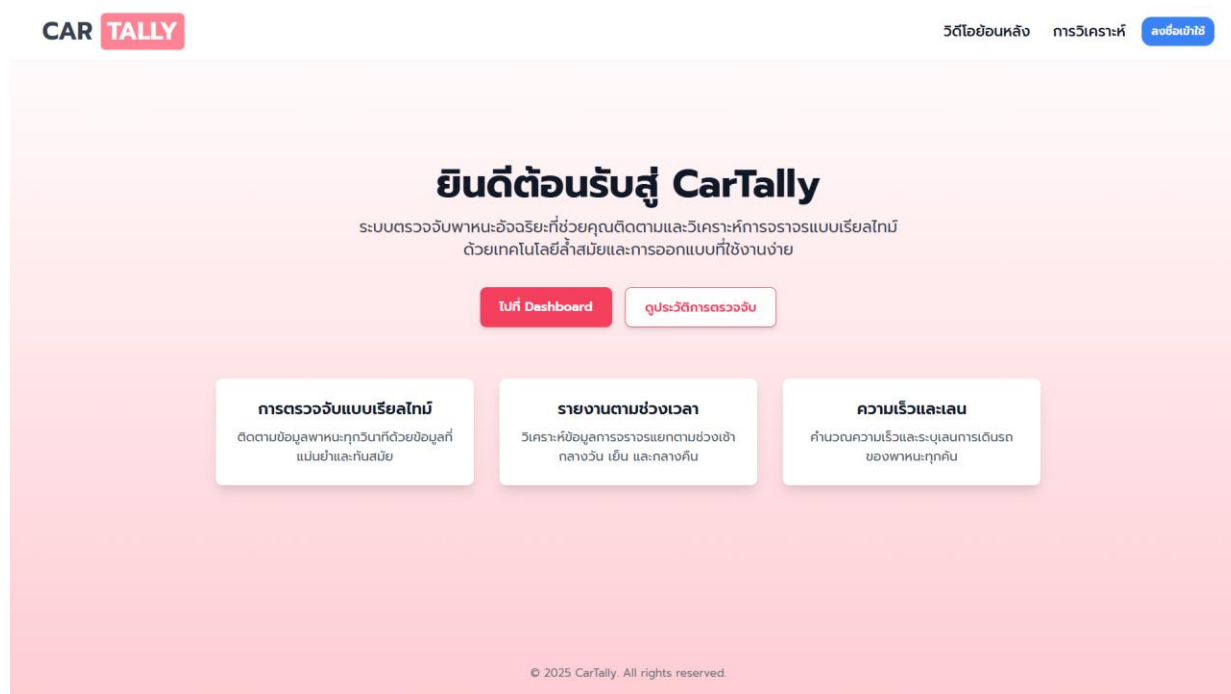
- **AI (Artificial Intelligence):** ใช้โมเดล YOLO (You Only Look Once) สำหรับการตรวจจับยานพาหนะและการคำนวณความเร็ว โดยมีการประมวลผลภาพผ่าน OpenCV โดยการทำงานของ AI จะถูกทำอยู่ใน localhost
- **Database:** ใช้ Cloud SQL ในการจัดเก็บข้อมูลยานพาหนะที่ตรวจพบ เช่น เวลาที่เข้าหรือออกจากเลน ความเร็วของรถ และข้อมูลอื่น ๆ ที่เกี่ยวข้อง



รูป 2 Diagram ภาพรวมของแอปพลิเคชัน

การทำงานฝั่ง Web Client

เว็บไซต์นี้พัฒนาขึ้นเพื่อรับข้อมูลจากกล้องที่ติดตั้ง แต่ในที่นี้จะใช้เป็นคลิปวิดีโอเนื่องจากไม่สามารถหา กล้องวงจรปิดที่อยู่ตามถนนได้ เพื่อสำหรับตรวจจับยานพาหนะโดยใช้เทคโนโลยีปัญญาประดิษฐ์ (AI) ในการ วิเคราะห์ภาพอย่างแม่นยำ ข้อมูลที่ได้จะถูกนำมาแสดงผลแบบเรียลไทม์ผ่านหน้าเว็บอย่างสวยงามและใช้งานง่าย และรองรับ Multiuser โดยผู้ใช้งานสามารถดูวิดีโอสดจากกล้อง พร้อมรายละเอียดของยานพาหนะที่ตรวจจับได้ เช่น จำนวนรถ, ประเภทของรถ (เช่น รถยนต์, รถบรรทุก), เวลาเข้าและออกจากระยะตรวจจับ นอกจากนี้ ระบบยัง คำนวณความเร็วของยานพาหนะจากระยะทางและเวลาที่บันทึกได้ แสดงผลในรูปแบบที่เข้าใจง่าย ช่วยให้ผู้ใช้ สามารถติดตามและวิเคราะห์การจราจร หากต้องการดูรายละเอียดการพัฒนา client [กดที่นี่](#)



รูป 3 เว็บไซต์หน้า Landing

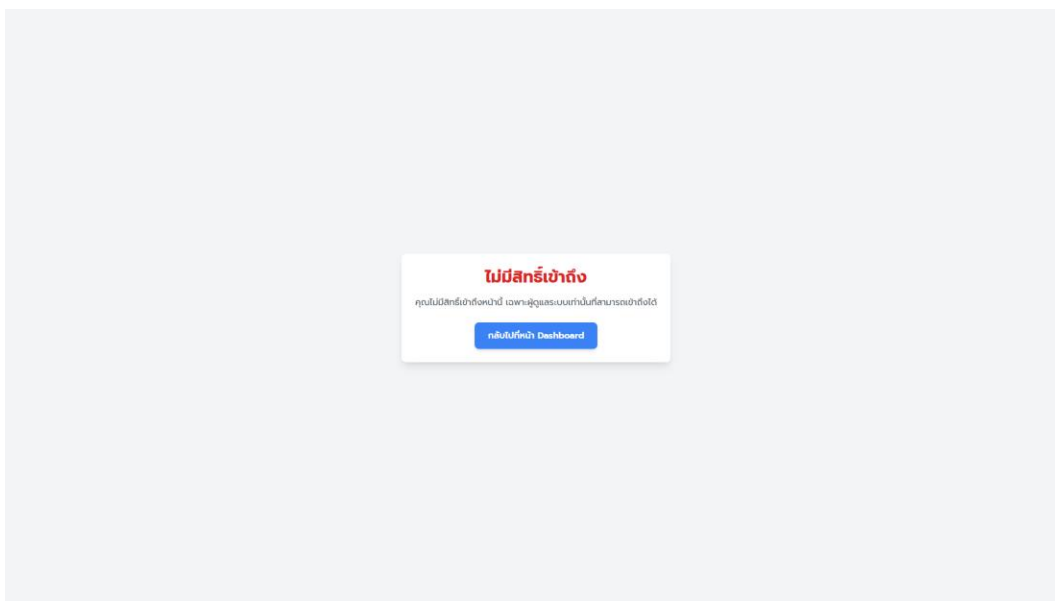
รูป 4 เว็บไซต์หน้าเข้าสู่ระบบ

รูป 5 เว็บไซต์หน้าลงทะเบียน

Multuser และ Security ในระบบ Login

ระบบนี้รองรับผู้ใช้หลายคน (Multuser) ในการล็อกอินพร้อมกัน โดยออกแบบให้สามารถจัดการ session และการเข้าถึงของผู้ใช้แต่ละคนได้อย่างปลอดภัยและมีประสิทธิภาพ ผู้ใช้จะต้องล็อกอินผ่านหน้า Login โดยระบุข้อมูลประจำตัว (เช่น อีเมลและรหัสผ่าน) ซึ่งจะถูกรับส่งไปยังเซิร์ฟเวอร์เพื่อตรวจสอบกับข้อมูลใน Cloud

SQL เมื่อการยืนยันสำเร็จ เซิร์ฟเวอร์จะสร้าง JSON Web Token (JWT) และส่งกลับไปยัง Client เพื่อเก็บใน sessionStorage และ cookies (เช่น ใน Navbar) JWT นี้ใช้สำหรับระบุตัวตนผู้ใช้ในทุก request ที่ส่งไปยัง API หรือ WebSocket ทำให้ระบบสามารถแยกแยะผู้ใช้แต่ละคนได้แม้จะใช้งานพร้อมกันหลายคนเพื่อกระจายข้อมูลเรียลไทม์ (เช่น วิดีโอและข้อมูลยานพาหนะ) ไปยังผู้ใช้ที่ล็อกอินทุกคน



รูป 6 การแสดงผลเว็บไซต์เมื่อไม่มีสิทธิ์เข้าถึง

หากเป็นผู้ใช้ที่ไม่ใช่ผู้ใช้ที่ได้รับอนุญาตพยายามเข้าหน้าที่กำหนด จะทำการแจ้งเตือนและย้ายมาสู่หน้านี้

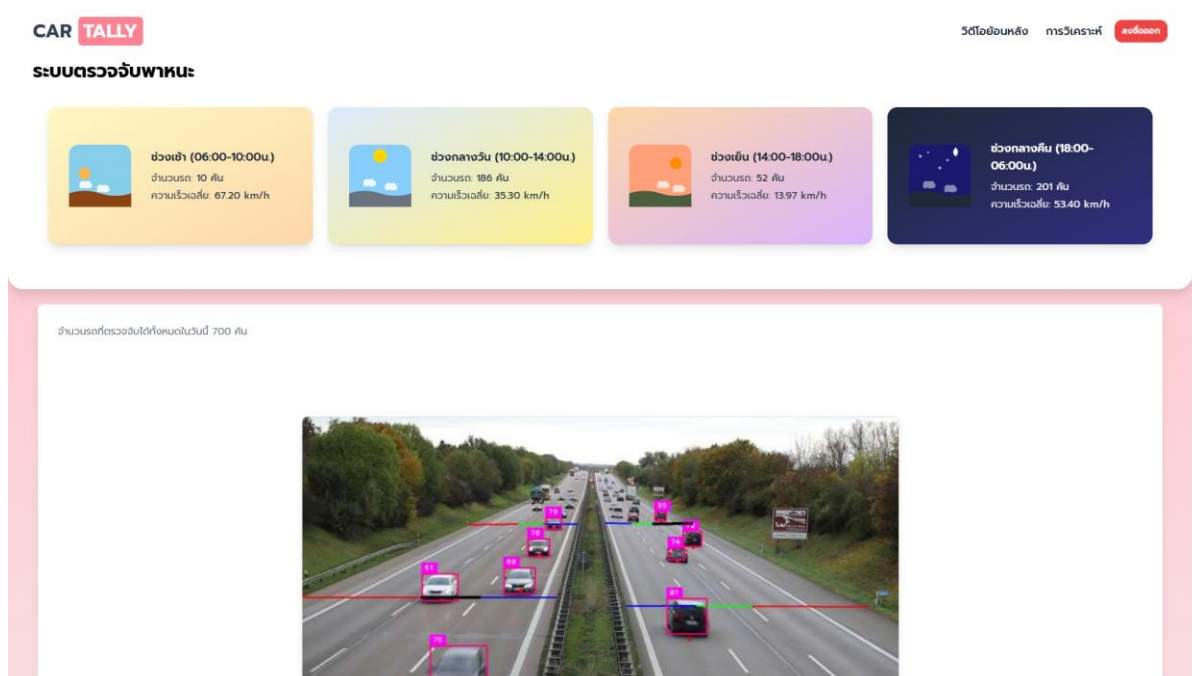
CAR TALLY 5 ตี้ออนหลัง การวิเคราะห์ ส่งอีเมล

รถที่ตรวจจบได้ล่าสุด ดูทั้งหมด

รายการแสดงรถที่ตรวจจบเรียงตามลำดับเวลา

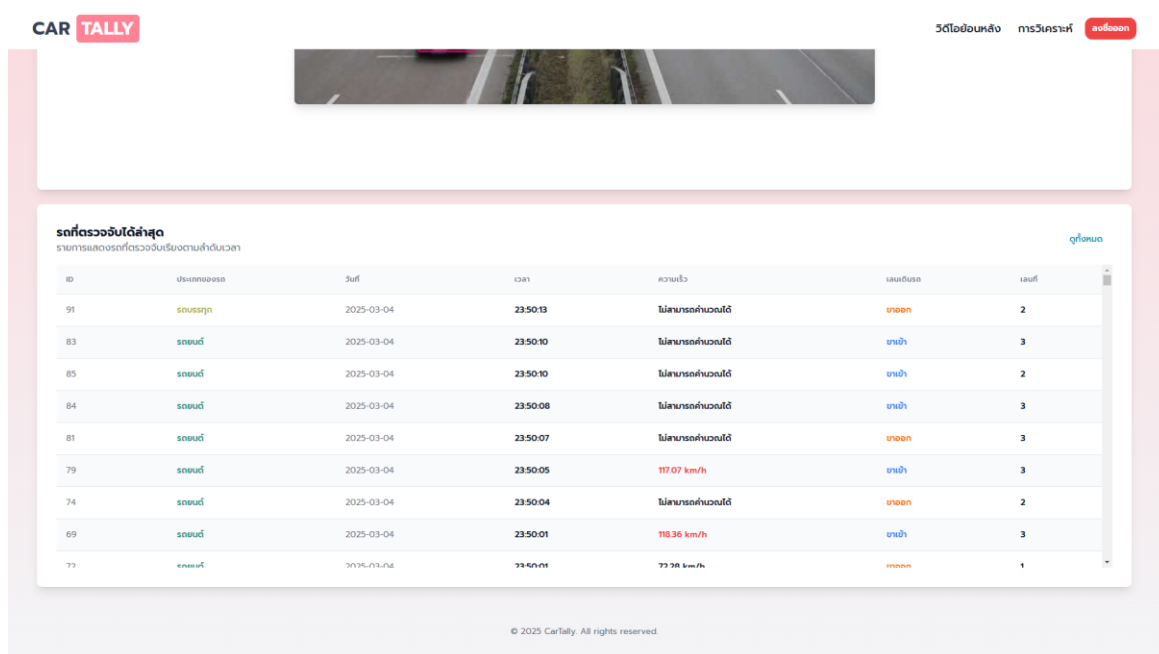
ID	ชื่อรถ	วันที่	เวลา
148	3ozLix1X	2025-03-04	00:38:05
154	C43Qld6v	2025-03-04	02:47:07
156	L0QskE1W	2025-03-04	15:58:39
163	pPmie6gt	2025-03-04	17:17:30

รูป 7 เว็บไซต์หน้าวิดีโอย้อนหลัง



รูป 8 หน้า Dashboard แสดงวิดีโอ

ในหน้านี้เราจะนำวิดีโอมาแสดง ทวีตวิดีโอที่ได้จากการตรวจจับของ AI จากนั้น นำมาแสดงในหน้าเว็บ ฝั่งเว็บจะทำการนำข้อมูลมา คำนวณสิ่งต่างๆ อาทิ เช่น พาหนะที่เข้าออก ตามช่วงเวลา ความเร็วรถเฉลี่ย ของแต่ละช่วงเวลา



รูป 9 หน้า Dashboard แสดง Log ของพาหนะที่ตรวจจับได้ล่าสุด

จากรูปด้านบน ข้อมูลที่ตรวจจับได้ล่าสุดมาแสดงแบบทันที พร้อมทั้ง คำนวณความเร็วแสดงออก ระบุทั้ง เวลา ชนิดของรถ และ เลนเดินรถ

CAR TALLY วิดีโอย้อนหลัง การวิเคราะห์ ลงข้อมูล

รถที่ตรวจจับได้ล่าสุด
รายการแสดงรถที่ตรวจจับเรียงตามลำดับเวลา

ID	ชื่อวิดีโอ	วันที่	เวลา
148	3outLixTX	2025-03-04	00:38:05
154	C43Of86v	2025-03-04	02:47:07
156	LOOskEiW	2025-03-04	15:58:39
163	pPme6gt	2025-03-04	17:17:30
164	4tAd9VcR	2025-03-04	17:49:58
165	M6UhgGU	2025-03-04	18:02:51
166	cl84t8bq	2025-03-04	18:03:22
167	LHbatLOB	2025-03-04	18:20:03
168	22piql6b	2025-03-04	18:20:38
169	eDZL1Bk	2025-03-04	18:21:16
170	HBTvY3zX	2025-03-04	19:05:37

รูป 10 หน้าวิดีโอย้อนหลังแสดงรายละเอียดวิดีโอล่าสุดและพาหนะที่ตรวจจับได้

CAR TALLY วิดีโอย้อนหลัง การวิเคราะห์ ลงข้อมูล

รถที่ตรวจจับได้ล่าสุด
รายการแสดงรถที่ตรวจจับเรียงตามลำดับเวลา

ID	ชื่อวิดีโอ	วันที่	เวลา	ความเร็ว	เลนเดินรถ	เลนที่
170	HBTvY3zX					
171	XLjxecGn					
172	dtppdphVM					
173	wOOv0qg					
174	ttt9qpb8y					
175	ttSBXYED					
176	Trt8xw97v					
177	P45pAs8					
178	twv8cFml					
179	KDMcTXcC					
180	XCddqweItt					

รายละเอียดวิดีโอ

ID	ประเภทของรถ	วันที่	เวลา	ความเร็ว	เลนเดินรถ	เลนที่
91	รถบรรทุก	2025-03-04	23:50:13	ไม่ทราบความเร็วได้ km/h	ขาออก	2
83	รถยนต์	2025-03-04	23:50:10	ไม่ทราบความเร็วได้ km/h	ขาเข้า	3
85	รถยนต์	2025-03-04	23:50:10	ไม่ทราบความเร็วได้ km/h	ขาเข้า	2
84	รถยนต์	2025-03-04	23:50:08	ไม่ทราบความเร็วได้ km/h	ขาเข้า	3
81	รถยนต์	2025-03-04	23:50:07	ไม่ทราบความเร็วได้ km/h	ขาออก	3
79	รถยนต์	2025-03-04	23:50:05	117.07 km/h	ขาเข้า	3
74	รถยนต์	2025-03-04	23:50:04	ไม่ทราบความเร็วได้ km/h	ขาออก	2
69	รถยนต์	2025-03-04	23:50:01	118.36 km/h	ขาเข้า	3
77	จักรยาน	2025-03-04	23:49:01	73.78 km/h	ขาออก	1

รูป 11 แสดงรายละเอียดวิดีโอ



รูป 12 หน้าการวิเคราะห์

ในหน้านี้เราจะนำข้อมูลรถที่ตรวจจับได้มาวิเคราะห์แสดงเป็นกราฟ เพื่อให้อยู่ในรูปแบบที่เข้าใจง่าย แสดงความสัมพันธ์ของ ช่วงเวลาย้อนหลัง 3 ชั่วโมง กับชนิดของรถ และ เลนเดินรถ

การทำงานฝั่ง Server และ Database

เซิร์ฟเวอร์นี้พัฒนาด้วย Express.js และทำงานบน Kubernetes โดยมีการออกแบบให้รองรับการทำงานแบบ Concurrency เพื่อให้สามารถรับคำขอจำนวนมากพร้อมกันได้อย่างมีประสิทธิภาพ เซิร์ฟเวอร์นี้ใช้ Prisma ORM ในการเชื่อมต่อฐานข้อมูล SQL ที่โฮสต์บน Google Cloud SQL และใช้ Docker เพื่อแพ็คเกจโค้ดให้สามารถรันได้บน Kubernetes Cluster ซึ่งถูกตั้งค่าให้ใช้ Load Balancer กระจายโหลดไปยังหลาย Instance ของเซิร์ฟเวอร์ได้อัตโนมัติ โดยมีรายละเอียดไฟล์ตาม [ลิงค์นี้](#)

API ของเซิร์ฟเวอร์ออกแบบเป็น RESTful API โดยแยกเป็นหลายโมดูลเพื่อความเป็นระเบียบและง่ายต่อการขยายและ ข้อมูลทั้งหมดถูกเก็บใน Cloud SQL โดยมีรายละเอียด API เบื้องต้น ดังนี้

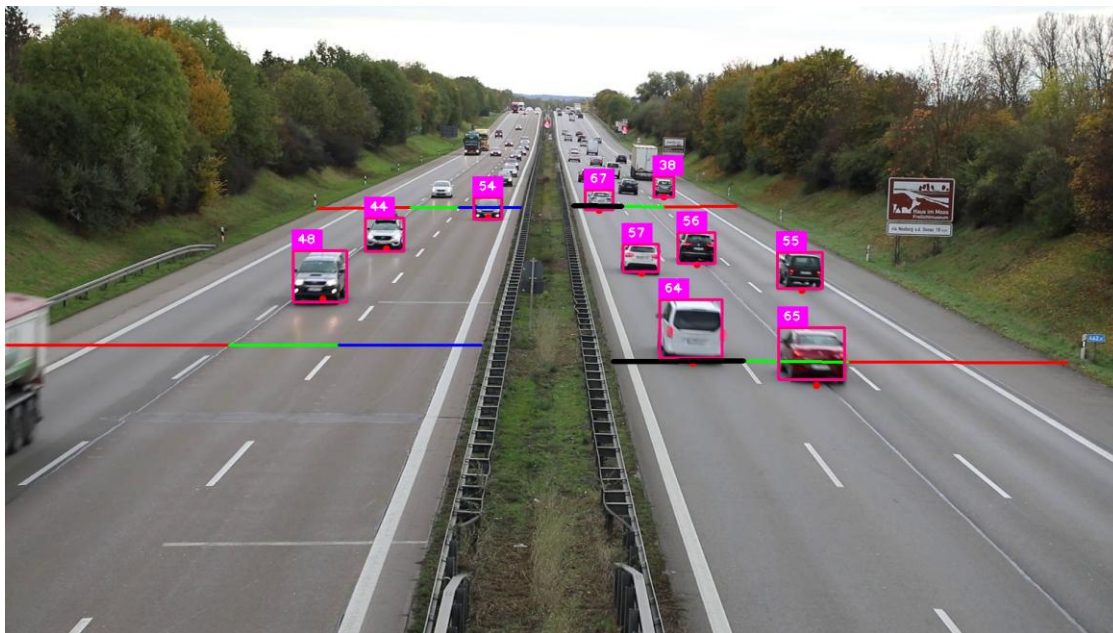
- /vehicle – ใช้จัดการข้อมูลยานพาหนะที่ถูกตรวจจับ
- /video – ใช้จัดเก็บและเรียกดูวิดีโอที่บันทึกไว้
- /user – ใช้สำหรับจัดการบัญชีผู้ใช้และการยืนยันตัวตน

เซิร์ฟเวอร์ใช้ Node.js Event Loop และ Asynchronous Processing เพื่อจัดการ Concurrency อย่างมีประสิทธิภาพ ลดการใช้ทรัพยากรโดยไม่ต้องสร้างเธรดเพิ่มเติมสำหรับแต่ละคำขอ Load Balancer ทำงานร่วมกับ Kubernetes Ingress เพื่อตรวจสอบและกระจายทราฟฟิกไปยังหลายๆ Replica ของแอปพลิเคชัน พร้อมเปิดใช้งาน Google Managed Certificate สำหรับรองรับ HTTPS และใช้ GKE Internal Load Balancer สำหรับการสื่อสารภายในคลัสเตอร์

แอปพลิเคชันถูกแพ็คเกจเป็น Docker Image และจัดการการ Deploy โดย Kubernetes Deployment โดยใช้ Rolling Update Strategy เพื่ออัปเดตแอปพลิเคชันโดยไม่หยุดให้บริการ นอกจากนี้ยังมีการตั้งค่า ConfigMap และ Secret ใน Kubernetes เพื่อเก็บค่าต่างๆ เช่น Database URL และคีย์ลับอย่างปลอดภัย

สำหรับการเชื่อมต่อฐานข้อมูล ระบบใช้ Google Cloud SQL (MySQL) ซึ่งสามารถเชื่อมต่อจาก Kubernetes Cluster พร้อมเปิดใช้งาน Automatic Backups และ Point-in-time Recovery เพื่อป้องกันข้อมูลสูญหาย

การทำงานฝั่ง AI

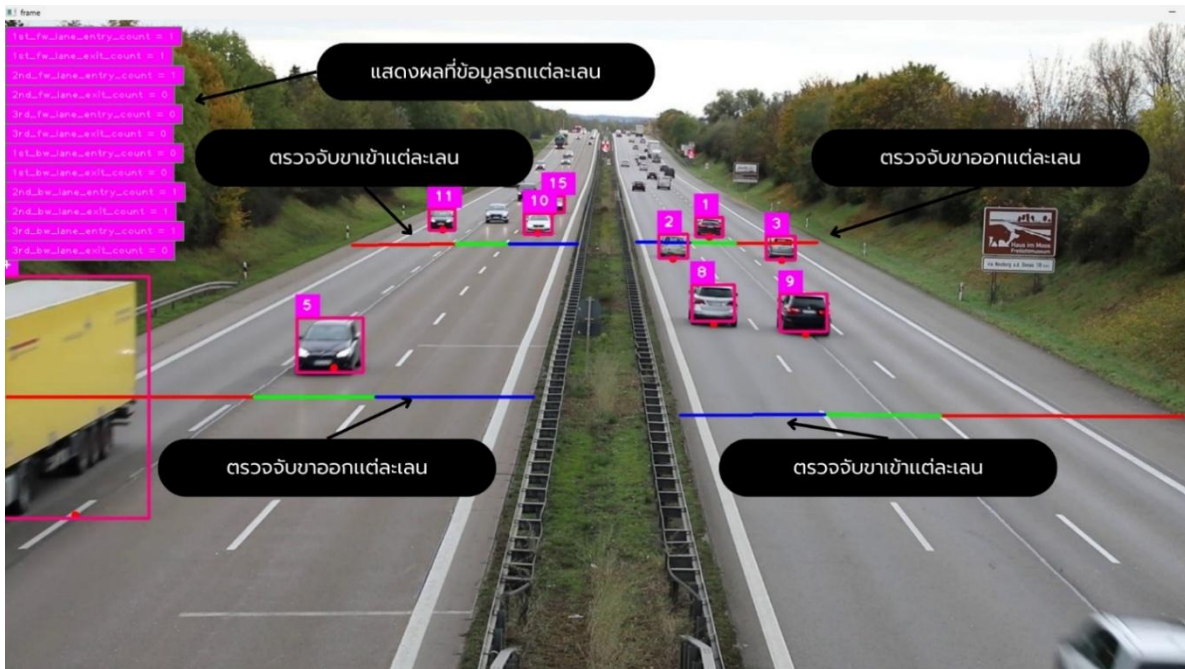


รูป 13 การแสดงผลเมื่อรัน main.py

ในส่วนของ AI ระบบถูกพัฒนาด้วย ภาษา Python โดยใช้โมเดลสำเร็จรูป YOLO สำหรับการตรวจจับยานพาหนะในวิดีโอ จากนั้นใช้ OpenCV เพื่อประมวลผลภาพและวาดกรอบล้อมรอบวัตถุที่ตรวจพบและแสดงรายละเอียดต่างๆ เช่น ประเภทรถยนต์, ตำแหน่ง และความเร็วของยานพาหนะ รวมถึงยังมีการใช้ไลบรารีอื่นๆ เพื่อเพิ่มประสิทธิภาพของการทำงาน เช่น requests, threading โดยโค้ดการทำงานเป็นไปดังลิงค์นี้ ลิงค์ [vehicle-detection-github](#) โดยไฟล์หลักๆมีการทำงานเป็นไป ดังนี้

- **main.py:** สคริปต์หลักที่ใช้สำหรับตรวจจับยานพาหนะในวิดีโอโดยใช้ OpenCV, YOLO และ cvzone โดยมีฟังก์ชันสำหรับเชื่อมต่อกับเซิร์ฟเวอร์ อ่านเฟรมจากวิดีโอ ตรวจจับวัตถุ ติดตามวัตถุ และส่งผลลัพธ์ไปยังเซิร์ฟเวอร์
- **services.py:** ไฟล์นี้มีฟังก์ชันสำหรับการทำ HTTP POST และ PUT requests ไปยังเซิร์ฟเวอร์ โดยกำหนด URL สำหรับสร้างและอัปเดตข้อมูลยานพาหนะและวิดีโอ รวมถึงใช้ threading เพื่อทำ asynchronous HTTP requests

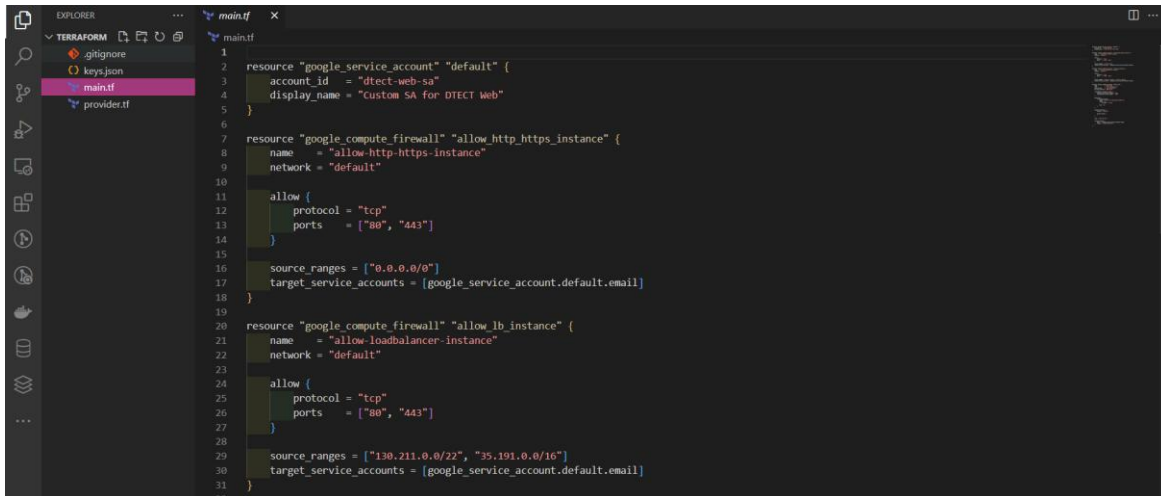
- **sort.py:** ไฟล์นี้เป็นการทำงานของ SORT (Simple Online and Realtime Tracking) algorithm สำหรับติดตามวัตถุที่ตรวจจับได้ โดยมีฟังก์ชันสำหรับการคำนวณ Intersection over Union (IoU) ระหว่าง bounding boxes และการติดตามวัตถุโดยใช้ Kalman filter
- **classes.txt:** ไฟล์ข้อความที่แสดงรายการคลาสของวัตถุที่สามารถตรวจจับได้โดย YOLO แต่ละบรรทัดแสดงถึงคลาสที่แตกต่างกัน เช่น "truck", "car" เป็นต้น



รูป 14 รายละเอียดการแสดงผลต่างๆเมื่อรัน main.py

การใช้งาน Cloud เพื่อ Deploy เว็บไซต์

- ขั้นตอนที่ 1 การสร้าง VM ผ่าน Terraform โดยรายละเอียดไฟล์ที่ใช้สร้าง VM เป็นไปตาม [ลิงค์นี้](#)

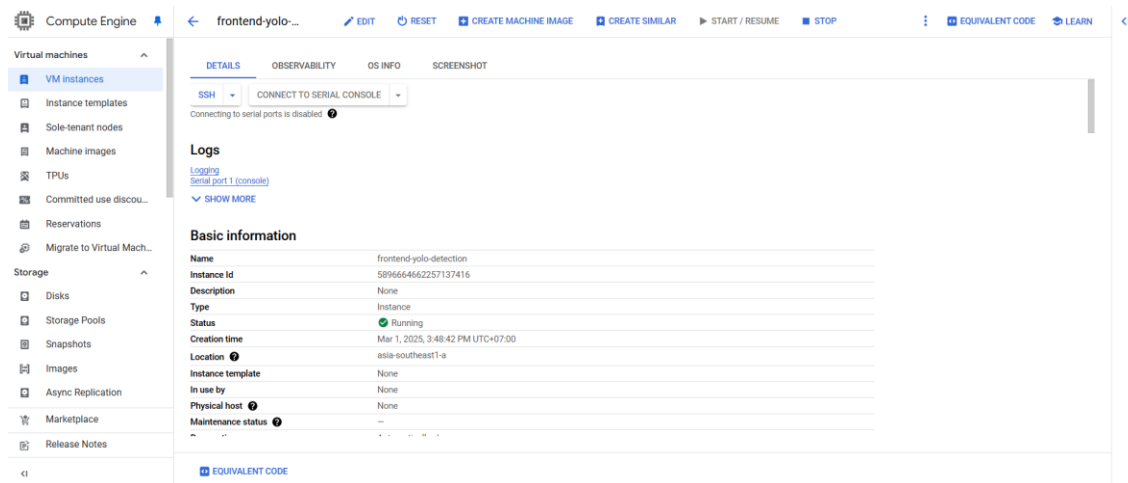


```

1
2 resource "google_service_account" "default" {
3   account_id = "dteet-web-sa"
4   display_name = "Custom SA for DTEET web"
5 }
6
7 resource "google_compute_firewall" "allow_http_https_instance" {
8   name = "allow-http-https-instance"
9   network = "default"
10
11   allow {
12     protocol = "tcp"
13     ports = ["80", "443"]
14   }
15
16   source_ranges = ["0.0.0.0/0"]
17   target_service_accounts = [google_service_account.default.email]
18 }
19
20 resource "google_compute_firewall" "allow_lb_instance" {
21   name = "allow-loadbalancer-instance"
22   network = "default"
23
24   allow {
25     protocol = "tcp"
26     ports = ["80", "443"]
27   }
28
29   source_ranges = ["130.211.0.0/22", "35.191.0.0/16"]
30   target_service_accounts = [google_service_account.default.email]
31 }

```

รูป 15 ไฟล์ main.tf ที่ใช้สร้าง VM



รูป 16 รายละเอียด VM ที่ได้จาก Terraform

- ขั้นตอนที่ 2 ลงแพ็คเกจต่างๆเบื้องต้นใน VM ผ่านคำสั่ง `sudo apt install nginx certbot python3-certbot-nginx` และลง `npm ,nodejs, pm2` เพื่อใช้สำหรับรัน Client
- ขั้นตอนที่ 3 ตั้งค่า config ใน `/etc/nginx/sites-available`

```

server {
    server_name dtect-sda-project.xyz;

    location /socket.io/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;

        add_header Content-Security-Policy "upgrade-insecure-requests";
        add_header Access-Control-Allow-Origin *;
        add_header Access-Control-Allow-Methods "GET, POST, OPTIONS";
        add_header Access-Control-Allow-Headers "Content-Type, Authorization, X-Requested-With";
    }

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
    }

    listen 443 ssl;
    ssl_certificate /etc/letsencrypt/live/dtect-sda-project.xyz/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/dtect-sda-project.xyz/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
}

```

รูป 17 ไฟล์ dtect ซึ่งเป็น config ของ nginx

- ขั้นตอนที่ 3 นำไฟล์โปรเจกต์ไปไว้ใน var/www/client และทำการ build ผ่านคำสั่ง npm run build

```

root@frontend-yolo-detection:/var/www# ls
client html
root@frontend-yolo-detection:/var/www# cd client/
root@frontend-yolo-detection:/var/www/client# ls
Dockerfile config jsconfig.json nginx.conf package-lock.json postcss.config.mjs src
README.md docker-compose.yml hooks next.config.mjs node_modules package.json public tailwind.config.mjs
root@frontend-yolo-detection:/var/www/client#

```

รูป 18 ข้อมูลหลังจาก build ใน /var/www/client

- ขั้นตอนที่ 4 รันโปรเจกต์ด้วยคำสั่ง pm2 start npm --name name_of_app -- start, pm2 save และ pm2 startup

```

Use --update-env to update environment variables
[PM2] Applying action restartProcessId on app [dtect](ids: [ 0 ])
[PM2] [dtect](0) ✓

```

id	name	namespace	version	mode	pid	uptime	o	status	cpu	mem	user	watching
0	dtect	default	0.39.1	fork	9071	0s	9	online	0%	22.0mb	root	disabled

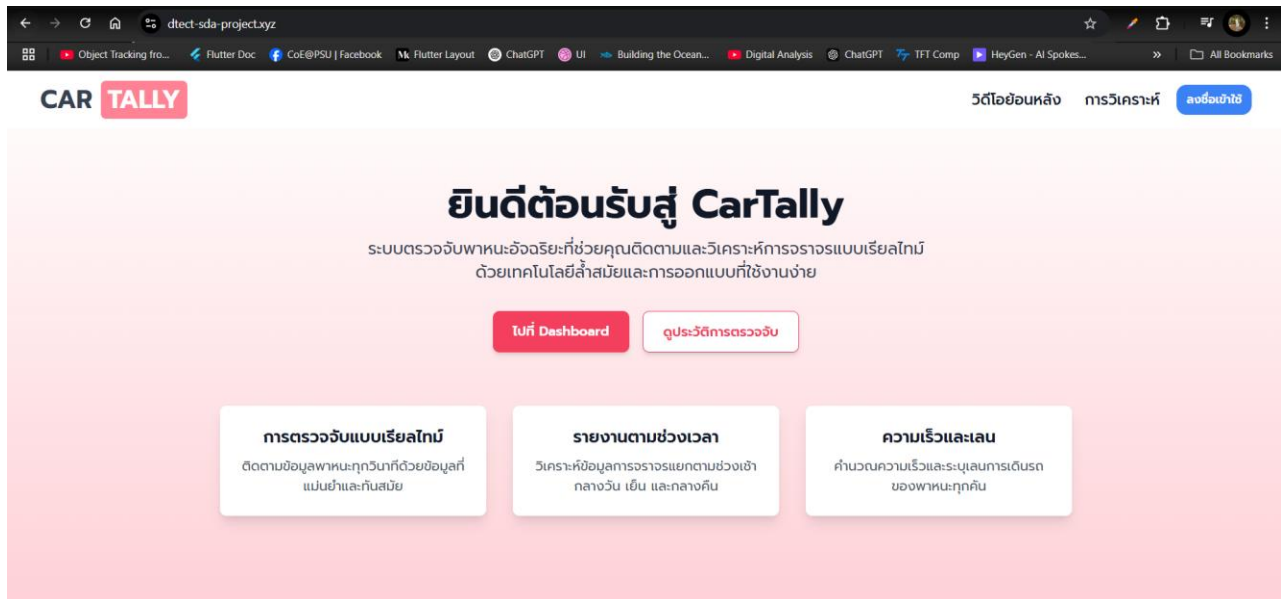
```

root@frontend-yolo-detection:/var/www/client#

```

รูป 19 การแสดงผลเมื่อใช้ pm2 ถูกต้อง

- ขั้นตอนที่ 5 ลง certbot เพื่อให้เว็บเป็น https ด้วยคำสั่ง sudo certbot --nginx -d dtect-sda-project.xyz -d www.dtect-sda-project.xyz



รูป 20 เว็บไซต์หลังจากทำทุกขั้นตอน

← Create a snapshot EQUIVALENT CODE

Snapshots are backups of persistent disks. They're commonly used to recover, transfer, or make data accessible to other resources in your project. [Learn more](#)

Name *
frontend-snapshot
Name is permanent

Description
snapshot for SDA Project

Snapshot source type *
Disk

Source disk *
frontend-yolo-detection

Type *

☒ Snapshot
Standard backup and disaster recovery; stored in a separate location from your disk

☐ Instant snapshot
Rapid restoration; stored in the same location as your disk

☐ Archive snapshot
Long-term storage for infrequently-accessed data; stored in a separate location from your disk

[COMPARE SNAPSHOT TYPES](#)

[CREATE](#) [CANCEL](#) [EQUIVALENT CODE](#)

รูป 21 รายละเอียดการสร้าง Snapshot ของ VM

Snapshots [CREATE SNAPSHOT](#) [CREATE SNAPSHOT SCHEDULE](#) [REFRESH](#) [DELETE](#)

Snapshots are backups of persistent disks. They're commonly used to recover, transfer, or make data accessible to other resources in your project. [Learn more](#)

[COMPARE SNAPSHOT TYPES](#)

SNAPSHOTS

ARCHIVE SNAPSHOTS

INSTANT SNAPSHOTS

NEW

SNAPSHOT SCHEDULES



Create snapshot schedules to automatically back up your data. [Learn more about creating snapshot schedules](#)

[DISMISS](#)

Filter Enter property name or value



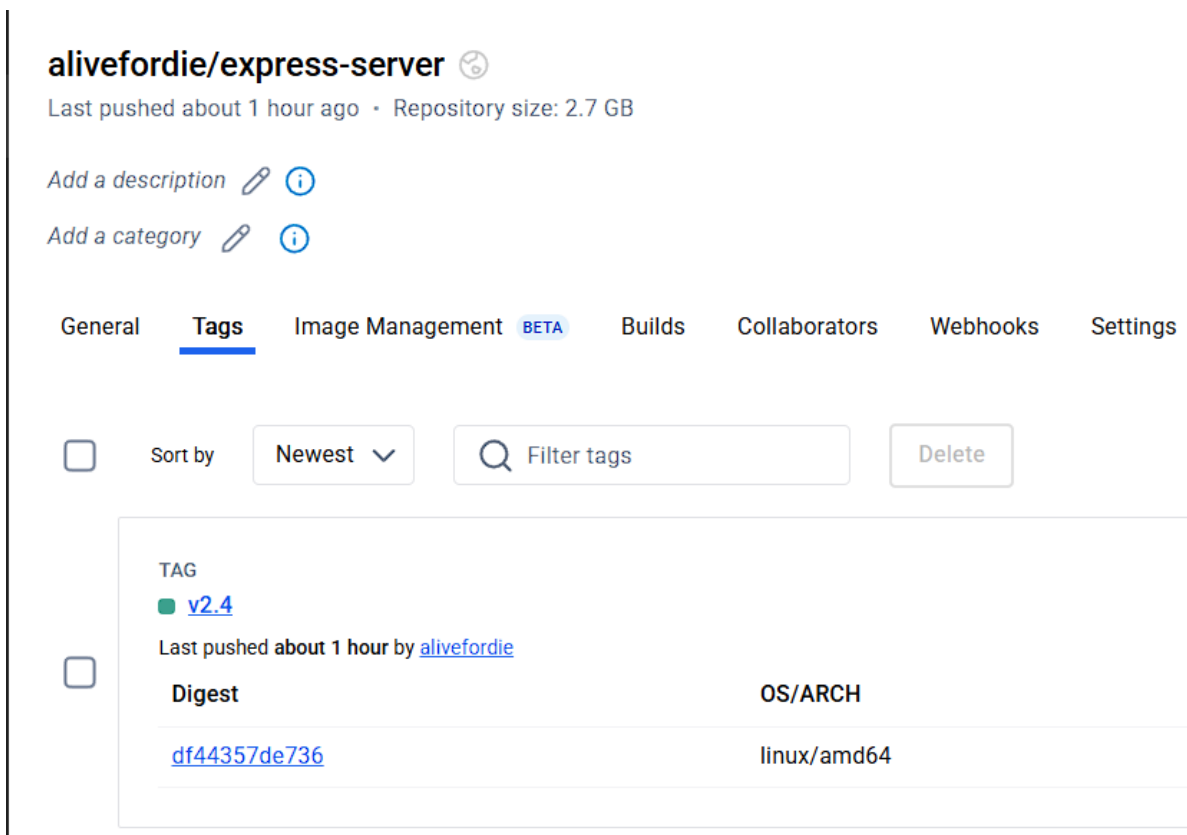
☐	Status	Name ↑	Location	Snapshot size	Creation time	Creation type	Source disk	Source instant snapshot
☐	✓	frontend-snapshot	asia	6.16 GB	Mar 5, 2025, 5:54:00 PM UTC+07:00	Manual	frontend-yolo-detection	

รูป 22 ผลการสร้าง Snapshot ของ VM

เมื่อทำการ Deploy เว็บไซต์เสร็จสิ้นและสามารถใช้งานได้แล้ว ต่อมาจะสร้าง Snapshot ของ VM ซึ่งเป็นขั้นตอนสำคัญเพื่อสำรองข้อมูลและป้องกันความเสียหายที่อาจเกิดขึ้นในอนาคต Snapshot ช่วยให้สามารถกู้คืนระบบกลับมาได้อย่างรวดเร็วหากเกิดข้อผิดพลาด รวมถึงใช้สำหรับทดสอบอัปเดตหรือเปลี่ยนแปลงระบบโดยไม่ต้องกังวลว่าข้อมูลเดิมจะสูญหาย นอกจากนี้ยังสามารถใช้ Snapshot เพื่อขยายระบบโดยการสร้าง VM ใหม่ที่มีการตั้งค่าเหมือนต้นฉบับ

การใช้งาน Cloud เพื่อ Deploy เซิร์ฟเวอร์

ใช้ Makefile เพื่อช่วยจัดการคำสั่งของ Docker และ Kubernetes ให้ทำงานได้สะดวกและรวดเร็วมากขึ้น โดยลดความซับซ้อนของการพิมพ์คำสั่งยาวๆ ซ้ำๆ ซึ่งสามารถกำหนด target สำหรับการ build, push, run container หรือ deploy ไปยัง Kubernetes cluster ได้ นอกจากนี้ยังรองรับการใช้ environment variables ทำให้การจัดการงานมีประสิทธิภาพมากขึ้น โดยมีรายละเอียดไฟล์ตาม [ลิงค์นี้](#)



รูป 23 Docker image ที่อยู่ใน Docker Hub

ในการ deploy เซิร์ฟเวอร์ ต้องทำการสร้าง Docker Image สำหรับการสร้าง container โดยใช้ไฟล์ Dockerfile ซึ่งกำหนดโครงสร้างและสภาพแวดล้อมของเซิร์ฟเวอร์เอาไว้ แล้วนำ Docker Image ที่สร้างขึ้นมานี้ไปเก็บไว้ที่ Container Registry Docker Hub เพื่อให้ Kubernetes Cluster สามารถดึงไปใช้งานได้

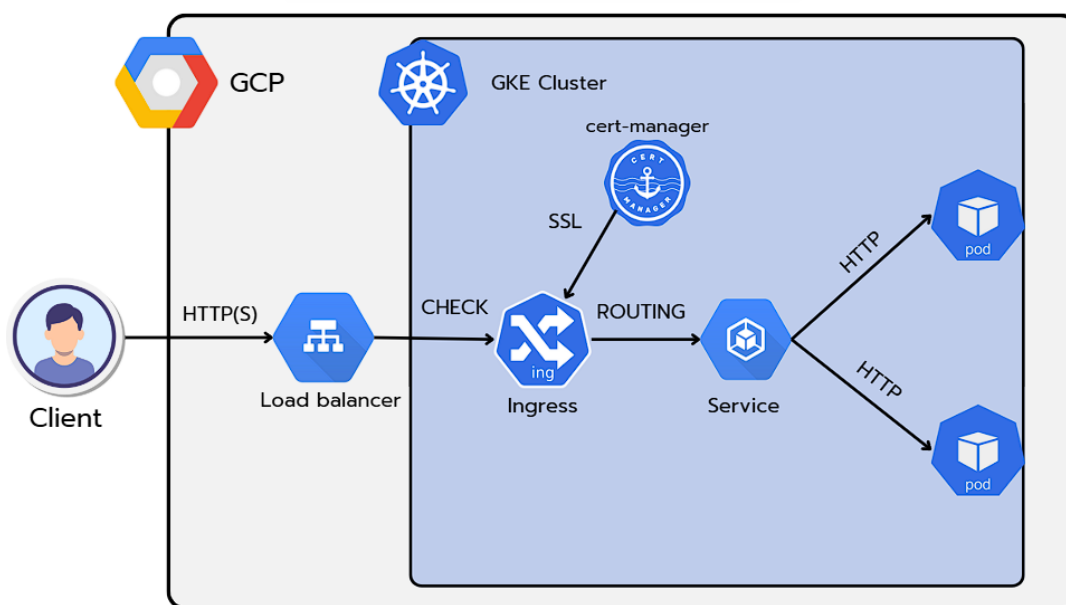
☐ Status	Name ↑	Location	Tier ⓘ	Number of nodes	Total vCPUs	Total memory	Notifications
☒	crush-on-k8s	asia-southeast1-a	Standard	3	6	12 GB	Set maintenance window

รูป 24 GKE Cluster

สร้าง Kubernetes Cluster บน Google Cloud เพื่อใช้เป็นโครงสร้างพื้นฐานในการรันเซิร์ฟเวอร์โดยกำหนดประเภทเครื่องเป็นรุ่นขนาดกลาง

```
gcloud container clusters get-credentials crush-on-k8s --zone=asia-southeast1-a --project=learn-vm-442206
```

จากนั้นตั้งค่าการเข้าถึง Kubernetes Cluster บนเครื่องที่ใช้งานอยู่ เพื่อให้สามารถใช้คำสั่ง kubectl และ makefile ที่เก็บคำสั่งของ kubectl เพื่อจัดการ Cluster ได้อย่างสะดวกมากขึ้น

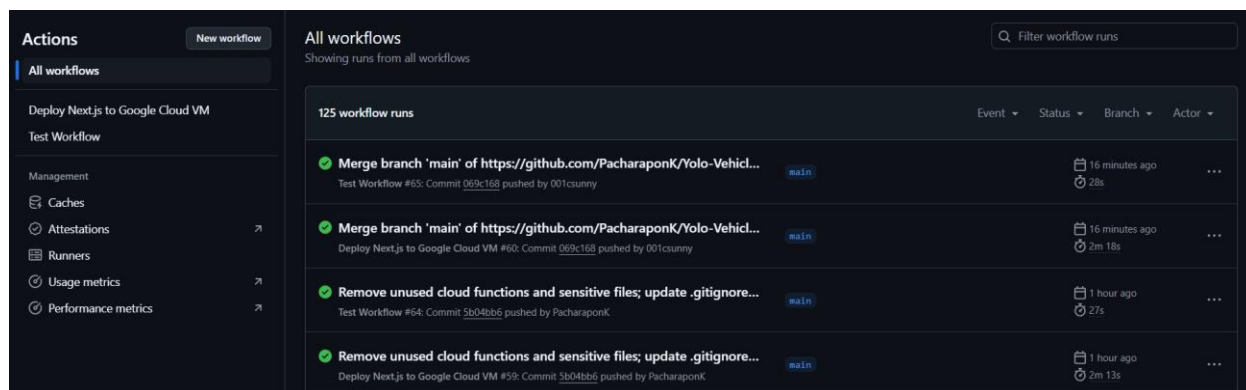


รูป 25 แผนภาพ Traffic ของเซิร์ฟเวอร์

ทำการ Deploy เซิร์ฟเวอร์บน Kubernetes โดยใช้ไฟล์กำหนดค่าต่างๆ all.yaml ซึ่งมีส่วนสำคัญดังนี้:

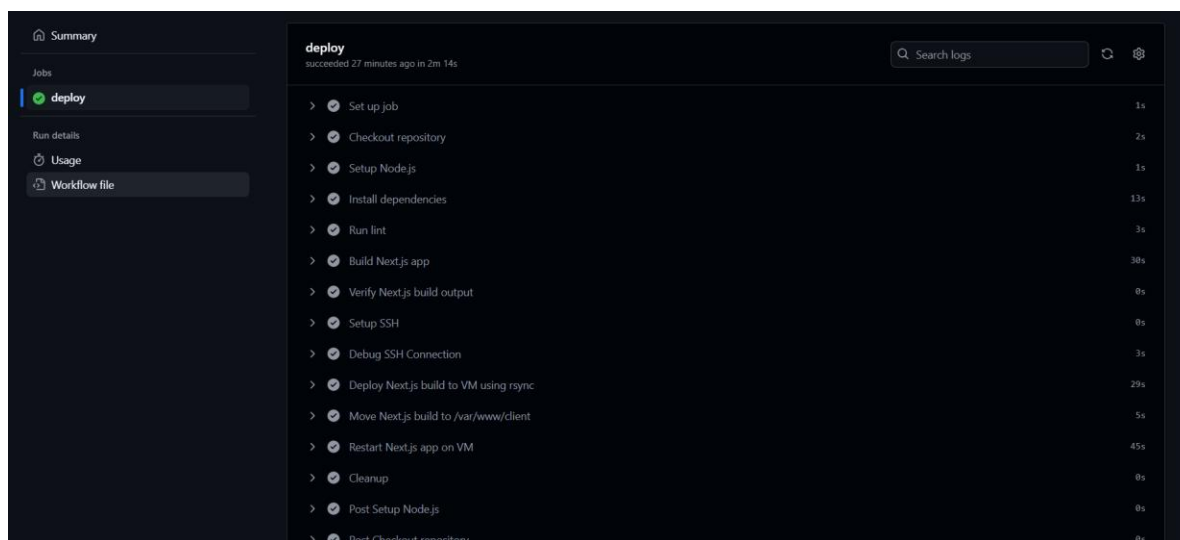
- **Load Balancer:** ช่วยกระจายโหลดของทราฟฟิกที่เข้ามายังเซิร์ฟเวอร์ ไปยัง ingress
- **Ingress:** กำหนดเส้นทางการเข้าถึงจากภายนอกผ่านโดเมนที่เข้ามาไป Service ที่ตรงกัน
- **Cert Manager:** ใช้ในการจัดการและออกใบรับรอง SSL/TLS โดยอัตโนมัติ เพื่อรองรับ HTTPS และเพิ่มความปลอดภัยให้กับแอปพลิเคชัน
- **Service:** กำหนดการเข้าถึงเซิร์ฟเวอร์ภายใน Cluster และรับส่งข้อมูลที่เข้า-ออก pods
- **Deployment:** กำหนดจำนวน pods (เซิร์ฟเวอร์) และ Docker Image ที่ใช้รัน

การใช้งาน Git Actions เพื่อทำ CI/CD

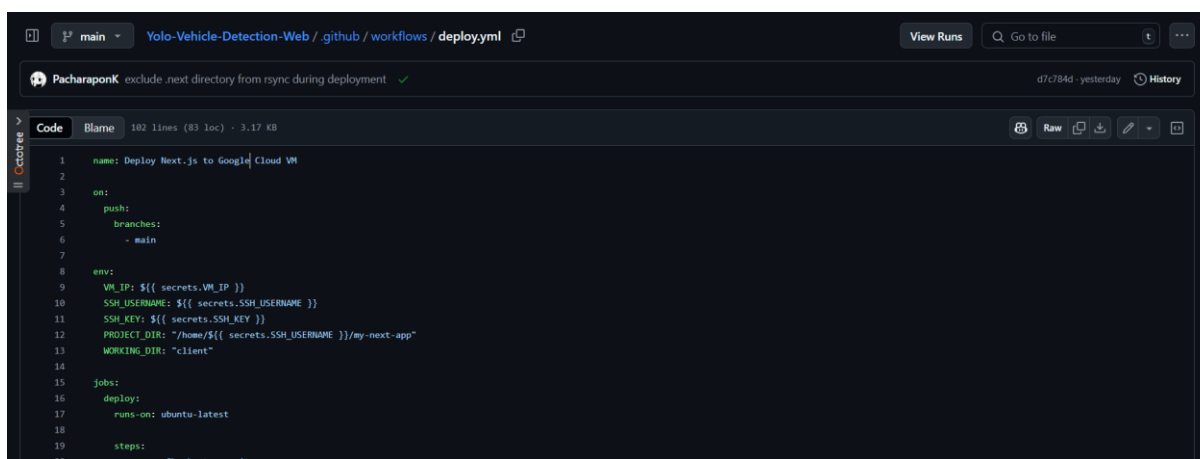


รูป 26 Git Actions ใน Repository

จากรูปมีการใช้งาน Git Actions สำหรับ CI/CD โดยมีด้วยกัน 2 workflows คือ Deploy Next.js to Google Cloud VM และ Test Workflow โดยแต่ละ Workflows มีการทำงานดังนี้



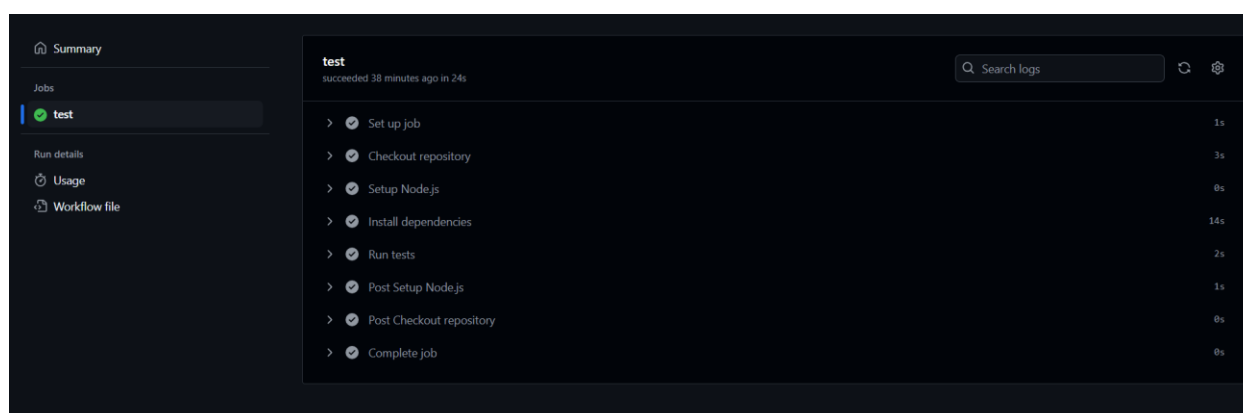
รูป 27 รายละเอียด Deploy Next.js to Google Cloud VM



รูป 28 ไฟล์ deploy.yml สำหรับ workflow

ไฟล์ deploy.yml เป็น GitHub Actions workflow ที่ใช้สำหรับการ Deploy แอปพลิเคชัน Next.js ไปยัง Google Cloud VM โดยอัตโนมัติเมื่อมีการ push ไปที่ branch main โดยมีไฟล์ตาม [ลิงค์นี้](#)

Workflow นี้มี ตัวแปร secrets ที่กำหนดค่าต่าง ๆ เช่น VM_IP, SSH_USERNAME, SSH_KEY เพื่อใช้ในการเชื่อมต่อ VM ผ่าน SSH เริ่มต้นด้วยการ checkout โค้ด, ติดตั้ง Node.js, ติดตั้ง dependencies, run lint, และ build แอป Next.js จากนั้นมีการ ตั้งค่า SSH และตรวจสอบการเชื่อมต่อ VM ก่อนใช้ rsync เพื่อคัดลอกโค้ดไปยังเซิร์ฟเวอร์ ถัดไปจะมีการ ย้ายไฟล์ build ไปยัง /var/www/client, ตั้งค่าความเป็นเจ้าของไฟล์, และ รีสตาร์ทแอปผ่าน pm2 สุดท้าย workflow จะทำความสะอาดไฟล์ ssh_key.pem เพื่อความปลอดภัย



รูป 29 รายละเอียด Test Workflow

```

1  name: Test Workflow
2
3  # เมื่อไหร่ workflow จะทำงาน
4  on:
5    push:
6      branches: [ main ]
7    pull_request:
8      branches: [ main ]
9
10 # งานที่ดำเนินการ
11 jobs:
12   test:
13     runs-on: ubuntu-latest
14
15     steps:
16     - name: Checkout repository
17       uses: actions/checkout@v3
18
19     - name: Setup Node.js
20       uses: actions/setup-node@v3
21       with:
22         node-version: '20'
23
24     - name: Install dependencies
25       run: npm install
26       working-directory: ./client
27
28     - name: Run tests
29       run: npm run lint
30       working-directory: ./client

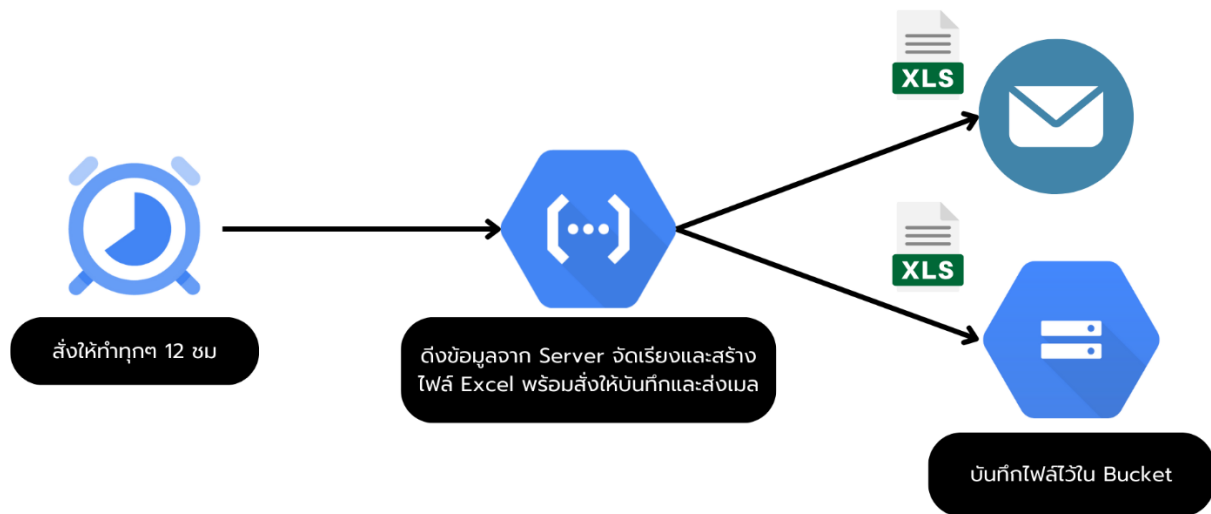
```

รูป 30 ไฟล์ test.yml สำหรับ workflow

ไฟล์นี้เป็น GitHub Actions Workflow ที่ใช้สำหรับ ทดสอบโค้ด (Test Workflow) โดยจะทำงานเมื่อมีการ push หรือ pull request ไปยัง branch main โดยมีรายละเอียดไฟล์ตาม [ลิงค์นี้](#)

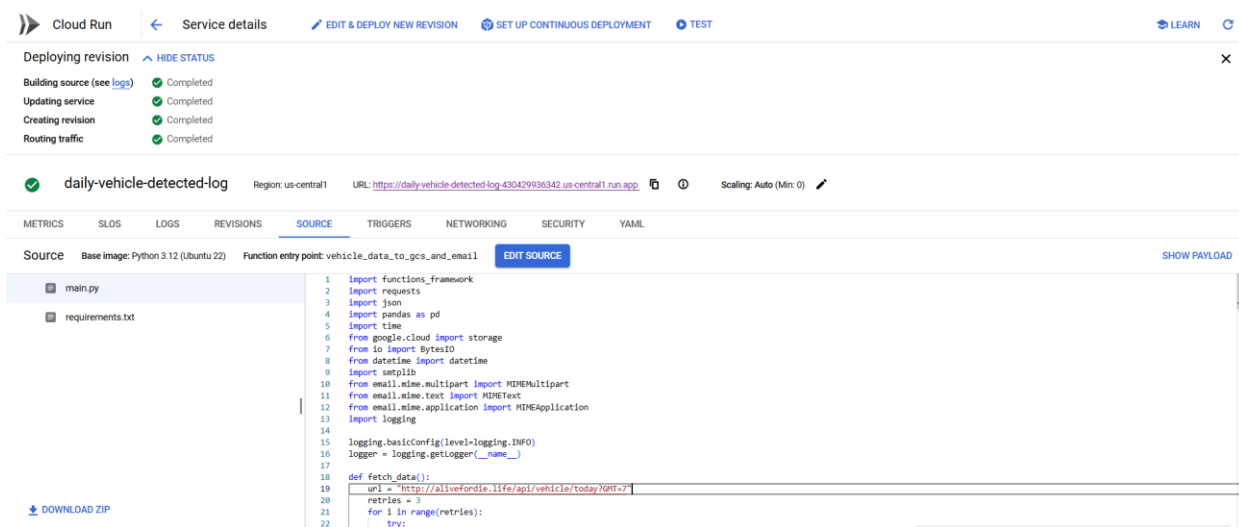
ภายใน workflow มี หนึ่ง job ชื่อว่า test ซึ่งรันบน Ubuntu ล่าสุด (ubuntu-latest) เริ่มต้นด้วย checkout โค้ด, ติดตั้ง Node.js เวอร์ชัน 20, ติดตั้ง dependencies ภายในโฟลเดอร์ client จากนั้นจะ รันคำสั่ง npm run lint เพื่อตรวจสอบคุณภาพของโค้ด (Linting) เพื่อให้แน่ใจว่าโค้ดไม่มีข้อผิดพลาดตามมาตรฐานที่กำหนด

การใช้งาน Cloud เพื่อส่งข้อมูลพาหนะรายวัน



รูป 31 แผนภาพการใช้งาน Cloud สำหรับการส่งข้อมูลพาหนะรายวัน

จากภาพจะมีการใช้งานเซอร์วิสต่างๆ ของ Google Cloud เพื่อใช้ในการส่งข้อมูลพาหนะรายวัน โดยเซอร์วิสที่ใช้ต่าง ๆ มีบทบาท ดังนี้



รูป 32 การใช้งาน Cloud Functions ชื่อ daily-vehicle-detected-log

- **Google Cloud Functions** เป็นฟังก์ชันที่ทำงานตามเหตุการณ์ (event-driven) ซึ่งถูกเรียกใช้งานเมื่อถึงเวลาที่กำหนดจาก Scheduler เพื่อประมวลผลข้อมูล Excel ของพาหนะรายวัน โดยรายละเอียดการทำงานนั้นจะเริ่มจากการดึงข้อมูลยานพาหนะจาก API แล้วนำมาประมวลผลก่อนอัปโหลดขึ้น Cloud Storage และส่งทางอีเมลเป็นไฟล์แนบ Excel โดยการทำงานเริ่มจากฟังก์ชัน `fetch_data()` ดึงข้อมูลจาก API และจัดการข้อผิดพลาดเบื้องต้น หากดึงข้อมูลสำเร็จ ฟังก์ชัน `export_to_gcs_and_get_buffer()` จะสร้าง DataFrame ด้วย pandas และปรับโครงสร้างข้อมูล เช่น แปลงค่าชื่อประเภทของยานพาหนะและรูปแบบเวลา จากนั้นไฟล์ Excel จะถูกสร้างและอัปโหลดไปยัง GCS ผ่าน `upload_to_gcs()` สุดท้าย ไฟล์เดียวกันนี้จะถูกส่งทางอีเมลโดย `send_email_with_attachment()` โค้ดนี้ใช้ Functions Framework เพื่อรองรับการเรียกผ่าน HTTP ทำให้สามารถใช้เป็น Web API ได้

The screenshot shows the Google Cloud Scheduler console for editing a job named 'vehicle-data_scheduler'. The job is configured with the following settings:

- Description:** daily-vehicle_scheduler
- Frequency:** 0 0/12 * * * (Every 12 hours)
- Timezone:** Indochina Time (ICT)
- Target type:** HTTP
- URL:** https://daily-vehicle-detected-log-430429936342.us-central1.run.app/
- HTTP method:** (Not specified in the image)

The console also includes a 'CONTINUE' button and a 'LEARN' link.

รูป 33 การใช้งาน Cloud Scheduler เพื่อส่งงานฟังก์ชัน

- **Google Cloud Scheduler** ทำหน้าที่เป็นเครื่องมือสำหรับตั้งเวลาการทำงานอัตโนมัติของระบบ ซึ่งในกรณีนี้ถูกกำหนดให้ทำงานทุกๆ 12 ชั่วโมง ระบบนี้จะส่งสัญญาณเริ่มต้นกระบวนการดึงและส่งข้อมูลพาหนะรายวัน โดยไปเรียกใช้ daily-vehicle-detected-log

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	ชื่อวัดโอ	รหัสรถ	ลำดับเลข	ชนิดรถ	เวลาที่ตรวจพบที่ตรวจจับ									
2	4SG7gpBJ	6	2	รถยนต์	18:30:12.5	2025-03-03								
3	4SG7gpBJ	7	1	รถบรรทุก	18:31:01.6	2025-03-03								
4	4SG7gpBJ	9	1	รถยนต์	18:30:28.4	2025-03-03								
5	4SG7gpBJ	12	3	รถยนต์	18:30:26.4	2025-03-03								
6	4SG7gpBJ	19	3	รถยนต์	18:30:31.7	2025-03-03								
7	4SG7gpBJ	23	2	รถยนต์	18:30:29.4	2025-03-03								
8	4SG7gpBJ	26	2	รถยนต์	18:30:40.6	2025-03-03								
9	4SG7gpBJ	32	2	รถยนต์	18:30:53.7	2025-03-03								
10	4SG7gpBJ	39	2	รถยนต์	18:31:20.5	2025-03-03								
11	4SG7gpBJ	41	2	รถยนต์	18:31:15.5	2025-03-03								
12	4SG7gpBJ	56	3	รถยนต์	18:31:27.1	2025-03-03								
13	4SG7gpBJ	63	2	รถยนต์	18:31:32.7	2025-03-03								
14	4SG7gpBJ	81	3	รถยนต์	18:31:35.5	2025-03-03								
15	4SG7gpBJ	88	3	รถยนต์	18:31:39.5	2025-03-03								
16	5fYpyGmJ	5	2	รถยนต์	18:04:11.6	2025-03-03								
17	5fYpyGmJ	10	3	รถยนต์	18:04:15.5	2025-03-03								
18	5fYpyGmJ	11	1	รถยนต์	18:04:16.7	2025-03-03								
19	5fYpyGmJ	12	2	รถยนต์	18:04:16.6	2025-03-03								
20	5fYpyGmJ	15	3	รถยนต์	18:04:17.3	2025-03-03								
21	5fYpyGmJ	23	2	รถยนต์	18:04:19.5	2025-03-03								
22	5fYpyGmJ	29	1	รถบรรทุก	18:04:26.6	2025-03-03								
23	5fYpyGmJ	30	2	รถยนต์	18:04:24.4	2025-03-03								
24	5fYpyGmJ	44	2	รถยนต์	18:04:33.7	2025-03-03								
25	5fYpyGmJ	48	2	รถยนต์	18:04:31.7	2025-03-03								
26	5fYpyGmJ	54	3	รถยนต์	18:04:34.6	2025-03-03								

รูป 34 ตัวอย่างไฟล์ Excel ที่ใช้ในการบันทึก

จากรูปด้านบนเป็นตัวอย่างไฟล์ Excel ที่สร้างจาก Cloud Functions

←

Bucket details

GO TO PATH

REFRESH

LEARN

dtect-bucket

Location

us (multiple regions in United States)

Storage class

Standard

Public access

Not public

Protection

Soft Delete

OBJECTS

CONFIGURATION

PERMISSIONS

PROTECTION

LIFECYCLE

OBSERVABILITY

INVENTORY REPORTS

OPERATIONS

Folder browser

dtect-bucket

data/

Buckets > dtect-bucket > data

CREATE FOLDER

UPLOAD

TRANSFER DATA

OTHER SERVICES

Filter by name prefix only

Filter

Filter objects and folders

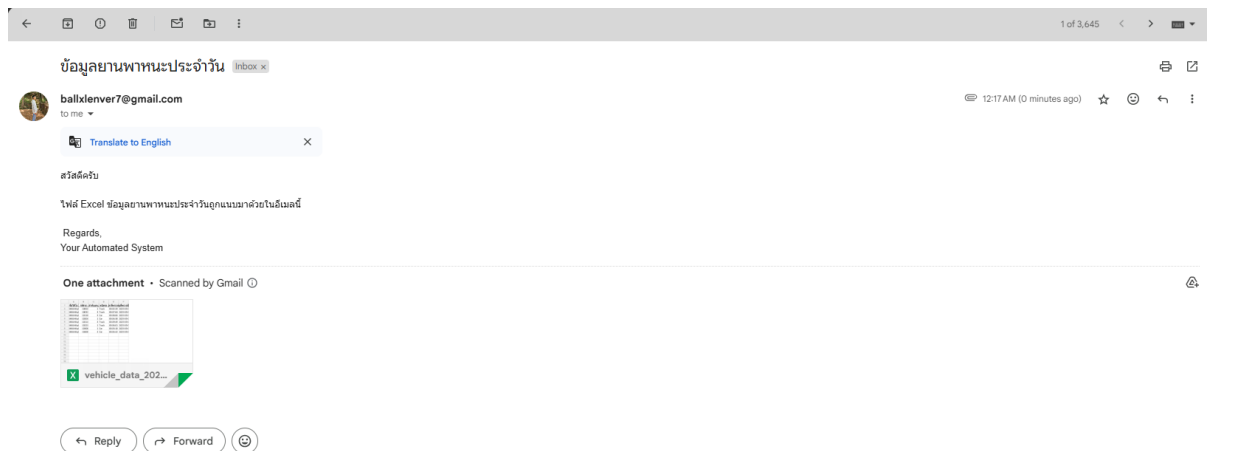
Show

Live objects only

☐	Name	Size	Type	Created	
☐	vehicle_data_20250303_133952.x_	16.2 KB	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	Mar 3, 2025, 8:39:54 PM	
☐	vehicle_data_20250303_155252.x_	18 KB	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	Mar 3, 2025, 10:52:54 PM	
☐	vehicle_data_20250303_170025.x_	6.3 KB	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	Mar 4, 2025, 12:00:27 AM	

รูป 35 Bucket ของ Google Cloud Storage

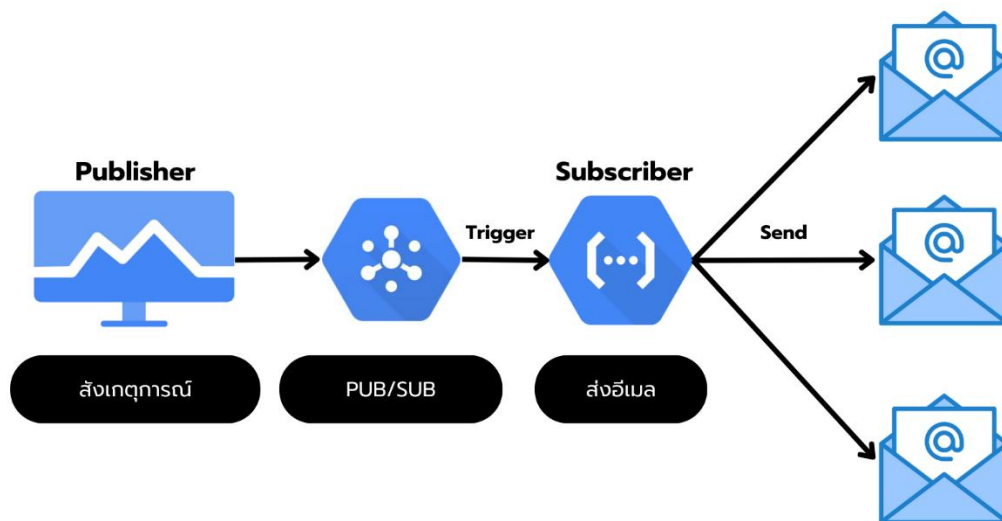
- Google Cloud Storage ทำหน้าที่เป็นที่เก็บข้อมูลในรูปแบบ Bucket ซึ่งใช้สำหรับอัปโหลดและเก็บไฟล์ Excel ของข้อมูลพาหนะรายวัน ข้อมูลที่เก็บไว้สามารถถูกเข้าถึงได้ทุกเมื่อผ่านอินเทอร์เน็ต และช่วยในการสำรองข้อมูลหรือวิเคราะห์ในอนาคต



รูป 36 ข้อมูลอีเมลที่ส่งให้ปลายทาง

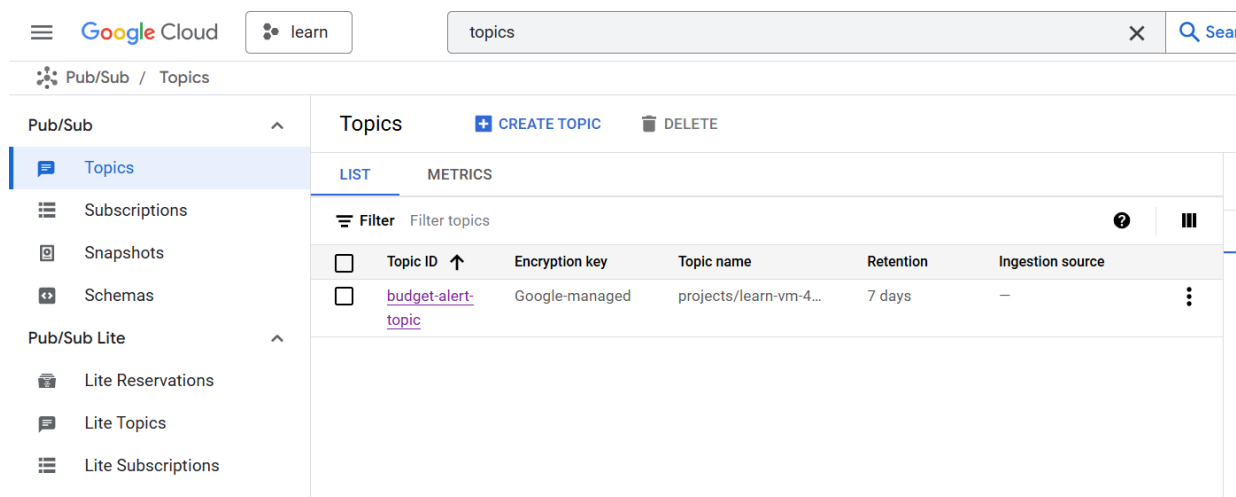
จากรูปด้านบนเป็นอีเมลโดยมีไฟล์ Excel ที่ส่งไปยังปลายทาง

การใช้งาน Cloud เพื่อสังเกตการณ์ค่าใช้จ่ายจากนั้นแจ้งเตือนไปยังอีเมล



รูป 37 แผนภาพแสดงลำดับการทำงานของ การส่งอีเมล

ระบบนี้ใช้ Google Cloud Platform (GCP) Monitoring เพื่อติดตามค่าใช้จ่ายของ Cloud โดยตั้งค่าให้ดึงข้อมูลการใช้งานทรัพยากร จากนั้นกำหนดนโยบายการแจ้งเตือน (alerting policy) ใน GCP Monitoring เพื่อตรวจสอบเมื่อค่าใช้จ่ายเกินเกณฑ์ที่กำหนด ข้อมูลการแจ้งเตือนจะถูกส่งไปยัง Pub/Sub topic ที่สร้างไว้ในชื่อ “budget-alert-topic” ผ่านการตั้งค่า notification channel เมื่อ Pub/Sub ได้รับข้อความ ระบบจะทริกเกอร์ Cloud Function ที่เขียนไว้เพื่อส่งอีเมลแจ้งเตือนไปยังผู้ดูแลระบบ



รูป 38 แสดง topic

Google Cloud Pub/Sub / Subscriptions / Subscription: budget-sub

budget-sub EDIT CREATE SNAPSHOT REPLAY MESSAGES PURGE MESSAGES DETACH DELETE HIDE INFO PANEL

Subscription name: projects/learn-vm-442206/subscriptions/budget-sub
 Subscription state: active
 Topic name: projects/learn-vm-442206/topics/budget-alert-topic

METRICS DETAILS MESSAGES

PULL ☐ Enable ack messages

Filter Filter messages

Publish time	Attribute keys	Message body	Ack ↑
Mar 2, 2025, 2:24:07 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 2:31:34 AM	—	{ "version": "test", "incid	Deadline exceeded
Mar 2, 2025, 2:44:09 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 3:04:23 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 3:24:26 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 3:51:00 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 4:11:06 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 4:44:38 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 5:10:50 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 5:31:01 AM	billingAccountId	{...}	Deadline exceeded
Mar 2, 2025, 6:04:09 AM	billingAccountId	{...}	Deadline exceeded

Filter Enter property name or value

Role / Principal ↑ Inheritance

- Cloud Build Service Agent (1)
- Cloud Dataflow Service Agent (1)
- Cloud Functions Service Agent (1)
- Editor (3)
- Eventarc Service Agent (1)
- Owner (3)

รูป 39 แสดงข้อมูลที่ถูกส่งรับผ่าน PUB/SUB

Google Cloud Billing Alerts <CloudPlatform-noreply@google.com>

50% of budget reached

Google Cloud OPEN CONSOLE

Billing Budget Alert

50% of budget reached

Billing account: Billing Account for Education Mar 3, 2025, 3:32 AM

Budget Amount	Budget Period
\$10.00	Mar 1, 2025 - Mar 31, 2025

Your billing account "Billing Account for Education" has reached 50% of the budget of \$10.00, for the budget period Mar 1, 2025 through Mar 31, 2025.

Budget Name: billing-alett

Billing Account ID: 016CEA-8BAGB3-EC2D84

Review your cost details in the [reports page](#)

VIEW BUDGET DETAILS

รูป 40 อีเมลที่ได้รับการแจ้งเตือน

การประมวลผลข้อมูลด้วย Multithreading เพื่อการจัดการยานพาหนะแบบเรียลไทม์

```

import requests
import threading

services = {
    "create_vehicle_url": "http://localhost:8080/api/vehicle/create",
    "update_vehicle_url": "http://localhost:8080/api/vehicle/update",
    "create_video_url": "http://localhost:8080/api/video/create"
}

def post_video(data):
    data = { "data" : data }
    response = requests.post(services["create_video_url"], json=data)
    return response.json()

def post_task(data):
    response = requests.post(services["create_vehicle_url"], json=data)
    return response.json()

def update_task(data, query):
    print(data, query)
    response = requests.put(services["update_vehicle_url"].format(query), json=data)
    return response.json()

def fire_and_forget(data):
    threading.Thread(target=post_task, args=(data,)).start()

def update_and_forget(data, query):
    threading.Thread(target=update_task, args=(data, query)).start()

```

รูป 41 โค้ด services สำหรับเรียกใช้ multithread

ระบบนี้พัฒนาขึ้นเพื่อรับข้อมูลจากกล้องที่ตรวจจับยานพาหนะโดยใช้ AI และส่งต่อไปยัง API เพื่อบันทึกและอัปเดตข้อมูล โดยใช้ Python และโมดูล threading เพื่อประมวลผลแบบ multithreading โค้ดประกอบด้วยฟังก์ชันหลัก 3 ส่วน

- post_video สำหรับบันทึกข้อมูลวิดีโอ
- post_task และ update_task สำหรับสร้างและอัปเดตข้อมูลยานพาหนะตามลำดับ
- fire_and_forget และ update_and_threading เพื่อส่งข้อมูลไปยัง API แบบ asynchronous โดยสร้าง thread แยกสำหรับการส่ง HTTP request ทำให้ main thread ไม่ต้องรอ response จาก API

และสามารถจัดการข้อมูลจากกล้องได้อย่างต่อเนื่อง ข้อมูลที่ส่งรวมถึงประเภทรถ, เวลาเข้า-ออก, และวิดีโอ ซึ่งเชื่อมโยงกับ frontend เพื่อแสดงผลแบบเรียลไทม์ การใช้ multithreading ซึ่งช่วยให้ระบบทำงานได้เร็วและมีประสิทธิภาพในสภาพแวดล้อมที่ต้องประมวลผลข้อมูลเรียลไทม์

สรุปรายการเทคนิคที่ใช้ในโปรเจกต์

Category I

- **Multithreading:** ใช้งาน threading เพื่อสร้าง thread สำหรับส่งข้อมูลไปยัง API แบบ asynchronous ดังที่กล่าวในหัวข้อ [การประมวลผลข้อมูลด้วย Multithreading เพื่อการจัดการยานพาหนะและวิดีโอแบบเรียลไทม์](#)
- **Concurrency:** เซิร์ฟเวอร์ใช้ Node.js Event Loop และ Asynchronous Processing เพื่อจัดการ Concurrency อย่างมีประสิทธิภาพ

Category II

- **Docker:** ใช้ Docker image สำหรับสร้าง pods ใน GKE Cluster
- **Cloud Functions:** ดึงข้อมูลยานพาหนะจาก API ประมวลผลด้วย Pandas สร้างไฟล์ Excel จากนั้นอัปโหลดไปยัง Cloud Storage และส่งทางอีเมลเป็นไฟล์แนบ อย่างที่กล่าวไปในหัวข้อ [การใช้งาน Cloud เพื่อส่งข้อมูลพาหนะรายวัน](#)
- **VM:** ใช้ Compute Engine สำหรับการสร้าง VM และ Deploy เว็บไซต์และสร้าง VM Cluster ของ Compute Engine สำหรับการ Deploy เซิร์ฟเวอร์

Category III

- **Terraform:** ใช้ในการสร้าง VM โดยมีรายละเอียดไฟล์ต่างๆ อย่างที่ได้กล่าวไปในหัวข้อ [การใช้งาน Cloud เพื่อ Deploy เว็บไซต์](#)
- **Git:** กลุ่มเราใช้ Git ในการทำ Version Control และใช้งาน Git Actions เพื่อทำ CI/CD อย่างที่ได้กล่าวไปในหัวข้อ [การใช้งาน Git Actions เพื่อทำ CI/CD](#)
- **Makefile:** ใช้เพื่อช่วยจัดการคำสั่งของ Docker และ Kubernetes ให้ทำงานได้สะดวกและรวดเร็วมากขึ้น โดยลดความซับซ้อนของการพิมพ์คำสั่งยาวๆ ซ้ำๆ อย่างที่ได้กล่าวไปในหัวข้อ [การใช้งาน Cloud เพื่อ Deploy เซิร์ฟเวอร์](#)

Category IV

- **Pub/Sub:** ใช้รับข้อมูลจากการสังเกตการณ์จากนั้นส่งนำข้อมูลกระจายแจ้งเตือนค่าบริการไปยังอีเมล อย่างที่กล่าวไว้ในหัวข้อ [การใช้งาน Cloud เพื่อสังเกตการณ์ค่าใช้จ่ายจากนั้นแจ้งเตือนไปยังอีเมล](#)
- **Load Balancer:** มีการใช้งาน Nginx ทำหน้าที่เป็นเว็บเซิร์ฟเวอร์และ Reverse Proxy ที่ช่วยจัดการทราฟฟิกในการ Deploy เว็บไซต์ โดยทำหน้าที่กระจายโหลดไปยังเซิร์ฟเวอร์ Backend หลายตัวเพื่อเพิ่มประสิทธิภาพและความเสถียรของระบบ นอกจากนี้ยังสามารถทำ SSL Termination ด้วย Certbot เพื่อเข้ารหัสการเชื่อมต่อผ่าน HTTPS และช่วยกำหนดเส้นทางการเข้าถึง (Routing) ไปยังเซิร์ฟเวอร์ที่เหมาะสมผ่านไฟล์ config ใน /etc/nginx/sites-available อย่างที่ได้กล่าวไปในหัวข้อ [การใช้งาน Cloud เพื่อ Deploy เว็บไซต์](#)
- **K8S:** เครื่องมือสำคัญที่ช่วยให้การบริหารจัดการแอปพลิเคชันใน Cloud มีความสะดวกและมีประสิทธิภาพมากขึ้น ไม่ว่าจะเป็นการบริหาร container, การจัดการทรัพยากร, การปรับขนาดอัตโนมัติ หรือการอัปเดตแอปพลิเคชัน Kubernetes ช่วยให้แอปพลิเคชันสามารถทำงานได้อย่างเสถียรและรองรับการขยายตัวในอนาคต
- **Scheduler:** ใช้งาน Cloud Scheduler ในการเป็นตัวสั่งงาน daily-vehicle-detected-log ที่อยู่ใน Cloud Functions ทุกๆ 12 ชั่วโมง อย่างที่กล่าวไปในหัวข้อ [การใช้งาน Cloud เพื่อส่งข้อมูลพาหนะรายวัน](#)

Category V

- **Snapshot:** สร้าง snapshot เพื่อสำรองข้อมูลของ VM อย่างที่กล่าวไปในหัวข้อ [การใช้งาน Cloud เพื่อ Deploy เว็บไซต์](#)
- **Backup:** ใช้ในการสำรองข้อมูลของ MySQL Database

- **Security:** การใช้ certbot เพื่อขอ HTTPS ทำให้ client/server รับ request จากที่เป็น HTTPS อย่างที่กล่าวไว้ในหัวข้อ [การใช้งาน Cloud เพื่อ Deploy เว็บไซต์](#) และ [Multiuser และ Security ในระบบ Login](#)

Category VI

- **Multiusers:** ระบบรองรับ Multiuser ในระบบ Login โดยออกแบบให้จัดการผู้ใช้หลายคนที่มีสิทธิ์การเข้าถึงแตกต่างกันได้อย่างปลอดภัย เพื่อป้องกันการเข้าถึงโดยไม่ได้รับอนุญาตและรักษาความปลอดภัยของข้อมูลผู้ใช้ อย่างที่กล่าวใน [Multiuser และ Security ในระบบ Login](#)
- **Other I (AI):** มีการใช้งาน Computer Vision เพื่อตรวจจับรถเข้าออกในแต่ละเลนโดยใช้ YOLO อย่างที่ได้กล่าวไปในหัวข้อ [การทำงานฝั่ง AI](#)
- **Other II (Monitoring and Alerting):** ใช้บริการ Cloud Monitoring เพื่อคอยตรวจสอบค่าใช้จ่ายของบริการต่างๆที่กำลังใช้งาน อย่างที่กล่าวไว้ในหัวข้อ [การใช้งาน Cloud เพื่อสังเกตการณ์ค่าใช้จ่ายจากนั้นแจ้งเตือนไปยังอีเมล](#)